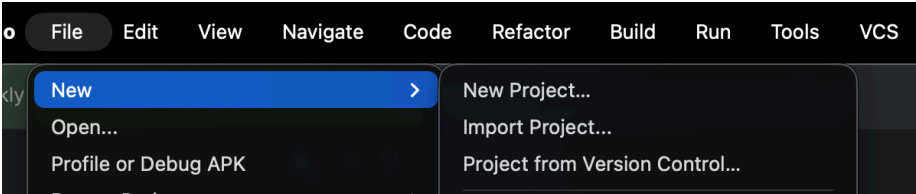


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Primeros pasos con Android Studio y Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	12/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Hanco Mullisaca Sergio Danilo - Huacani Jara Denise Andrea				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
I. SOLUCION DE PROBLEMAS - Link Repositorio (Sergio): https://github.com/shanccom/IDNP_Repo/tree/main/lab_02_Hello_World - Link Repositorio (Denise): https://github.com/DeniseHuacani/IDNP-LABORATORIO/blob/main/app/src/main/java/com/example/helloworldcompose/MainActivity.kt Propuesta 1 Ejercicio 1: Replicar el ejercicio resuelto por el docente (proyecto Compose + Hello World). 1. Abrir Android Studio. En la pantalla de bienvenida, seleccionar "New Project".  Figura 1: New Project 2. En la lista de plantillas, elegir "Empty Activity" (dentro de la categoría Phone and Tablet). Esta plantilla ya viene configurada con Jetpack Compose.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

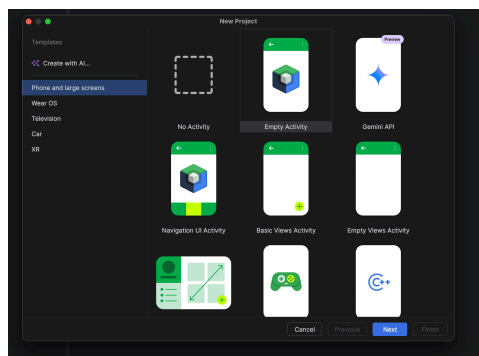


Figura 2: Empty Activity

- Configurar el nombre del proyecto (por ejemplo, "HelloWorldCompose"), el nombre del paquete y confirmar que el lenguaje sea Kotlin. Presionar "Finish".

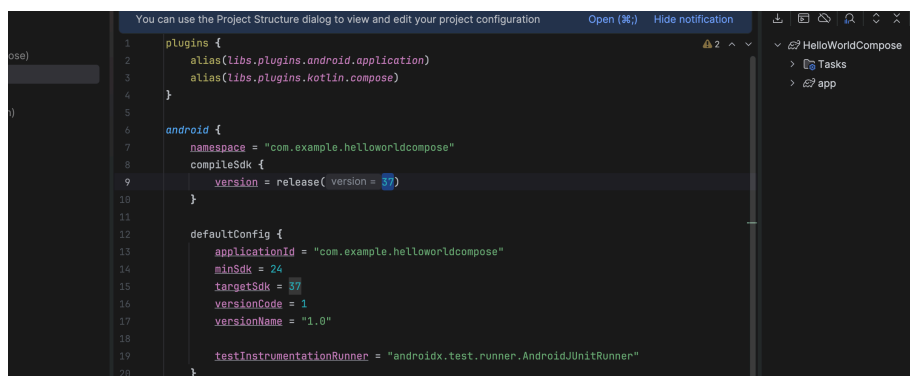


Figura 3: Project Configuration

- Esperar a que Android Studio descargue dependencias y sincronice Gradle. Se abrirá automáticamente el archivo MainActivity.kt con código ya generado.

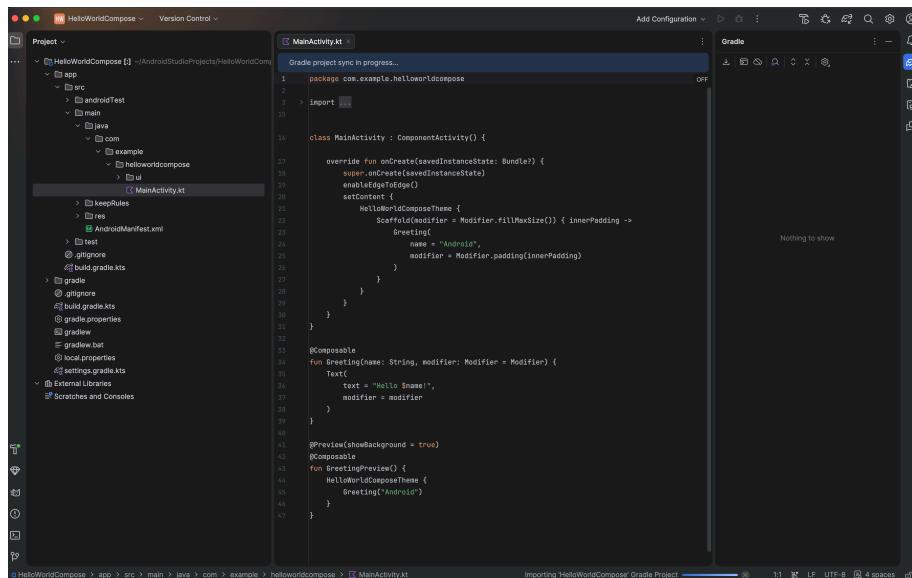


Figura 4: Project Created

5. Ejecutar el proyecto presionando el botón Run ► (con un emulador o un dispositivo físico conectado). Debe aparecer el mensaje "Hello Android!" en pantalla.

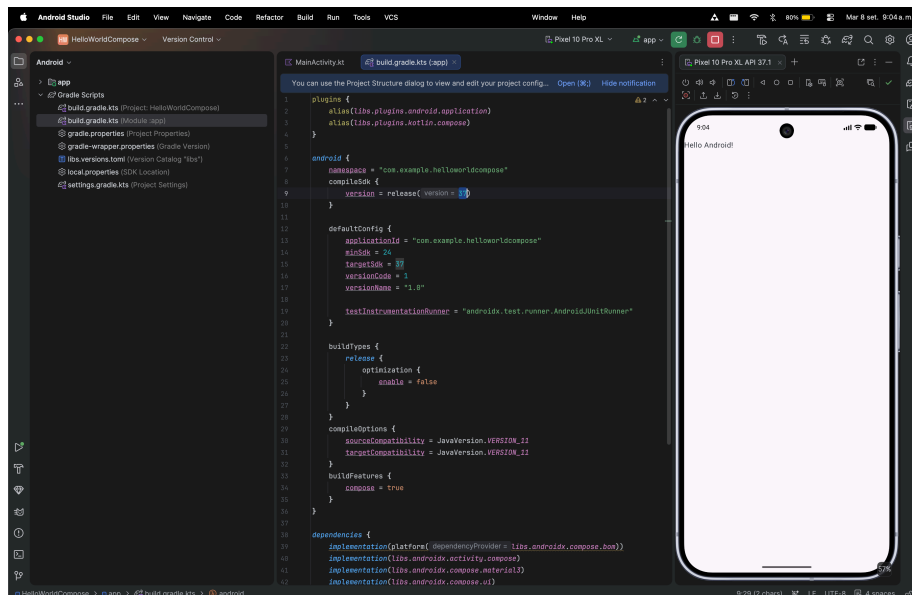


Figura 5: Hello Android

6. Dentro de MainActivity.kt, ubicar la función @Composable llamada Greeting(name: String, ...). Esta función recibe un texto y lo muestra mediante un componente Text.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

2 Usages
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "    Hello $name!",
        modifier = modifier
    )
}

```

Figura 6: New Project

- Ubicar la llamada `Greeting(name = "Android", ...)` dentro de `setContent { }`, y reemplazar el texto "Android" por otro (por ejemplo, el nombre del estudiante).

```

1 Usage
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            HelloWorldComposeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding( paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}

```

Figura 7: New Project

- Volver a ejecutar Run y verificar que el emulador ahora muestra el saludo personalizado.

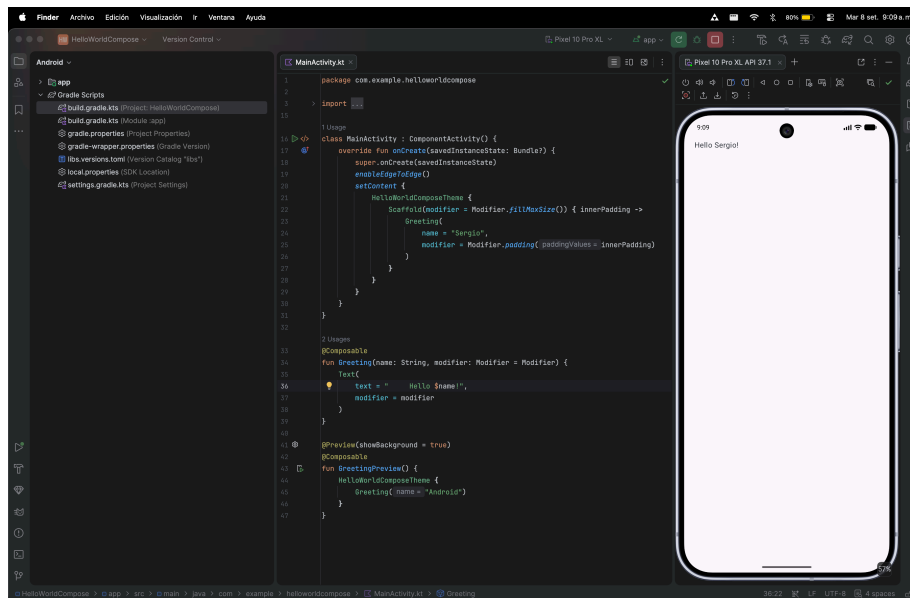


Figura 8: Hello Sergio

Ejercicio 2: Una pantalla (Composable) con tres campos de texto (TextField): título del libro, autor y número de páginas leídas, y dos botones: “Guardar” y “Ver registro”.

El objetivo de este ejercicio es construir la interfaz de registro de lectura de la aplicación: una pantalla creada con Jetpack Compose que contiene tres campos de texto para capturar los datos de un libro y dos botones para guardarlos y consultarlos más adelante.

1. Crear el Composable RegistroLibro

Dentro de MainActivity.kt, se define un Composable llamado RegistroLibro() que se carga en setContentView { } al iniciar la actividad. Es el encargado de dibujar toda la pantalla del formulario:

```
setContent {
    RegistroLibro()
}
```

2. Declarar las variables de estado

Para que cada campo recuerde lo que el usuario escribe, se usan variables de estado con remember { mutableStateOf("") }. Se necesita una variable por dato: título, autor y páginas. Además, registro almacenará el texto leído del archivo cuando se pulse “Ver registro”:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```
val context = LocalContext.current

var titulo by remember { mutableStateOf("") }
var autor by remember { mutableStateOf("") }
var paginas by remember { mutableStateOf("") }
var registro by remember { mutableStateOf("") }
```

3. Agregar los tres campos de texto

Cada dato se captura con un `OutlinedTextField`. Este componente recibe el valor actual en `value` y actualiza la variable de estado en `onValueChange`, es decir, cada vez que el usuario escribe se vuelve a renderizar el campo con el texto nuevo:

- Título del libro vinculado a `titulo`.
- Autor vinculado a `autor`.
- Páginas leídas vinculado a `paginas`.

4. Agregar los dos botones

Debajo de los campos se colocan dos `Button` que ocupan todo el ancho de la pantalla:

- GUARDAR: en esta etapa solo invoca a la función que almacena los datos (se detalla en el siguiente ejercicio).
- VER REGISTRO: llamará a la función que lee el archivo guardado.

El siguiente fragmento muestra las variables de estado, los tres `OutlinedTextField` y los dos botones:

```
val context = LocalContext.current

var titulo by remember { mutableStateOf("") }
var autor by remember { mutableStateOf("") }
var paginas by remember { mutableStateOf("") }

OutlinedTextField(
    value = titulo,
    onValueChange = { titulo = it },
    label = { Text("Título del libro") }
)

OutlinedTextField(
    value = autor,
    onValueChange = { autor = it },
    label = { Text("Autor") }
)

OutlinedTextField(
    value = paginas,
    onValueChange = { paginas = it },
    label = { Text("Páginas leídas") }
)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

Button(onClick = { GuardarDatos(context, titulo, autor, paginas) }) {
    Text("GUARDAR", fontWeight = FontWeight.Bold)
}

Button(onClick = { registro = LeerDatos(context) }) {
    Text("VER REGISTRO", fontWeight = FontWeight.Bold)
}

```

5. Ejecutar y verificar

Al ejecutar el proyecto con un emulador o un dispositivo físico conectado se muestra la pantalla con el formulario completo: tres campos de texto y los dos botones “Guardar” y “Ver registro”.

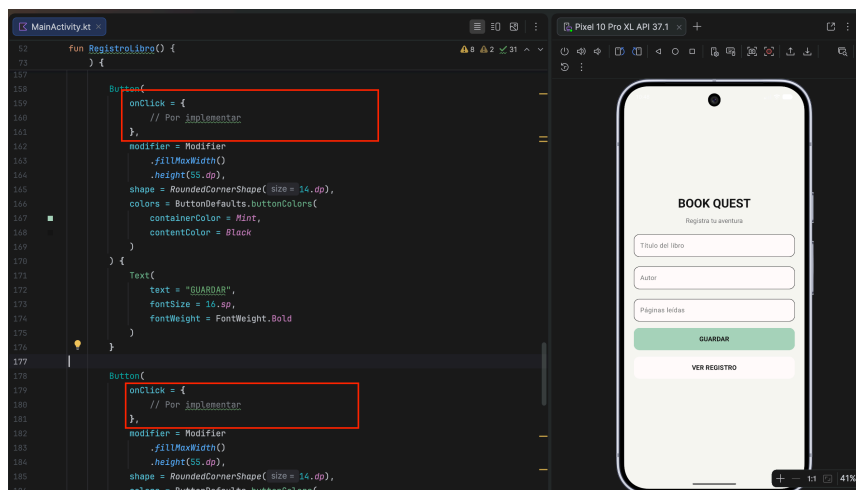


Figura 9: Pantalla con Campos de texto

Ejercicio 3: Al presionar “Guardar”, escribir los tres datos ingresados en un archivo de texto plano guardado en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_PRIVATE).

Al presionar el botón “Guardar” se ejecuta GuardarDatos(context, titulo, autor, paginas), que recibe los tres datos capturados en los campos de texto. Dentro de la función se construye la cadena del registro y se usa context.openFileOutput("datos.txt", Context.MODE_PRIVATE) para crear (o abrir) el archivo en el almacenamiento interno en modo privado, escribiendo el contenido con archivo.write(datos.toByteArray()).

```

fun GuardarDatos(
    context: Context,
    titulo: String,
    autor: String,

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
paginas: String,
) {
    val datos = "Titulo: $titulo Autor: $autor paginas: $paginas".trimIndent()

    context.openFileOutput(
        "datos.txt",
        Context.MODE_PRIVATE
    ).use { archivo ->
        archivo.write(datos.toByteArray())
    }
}
```

Al ejecutar el proyecto e ingresar los datos, al presionar “Guardar” se crea el archivo datos.txt en el almacenamiento interno de la aplicación:

```
1 Usage
fun GuardarDatos(
    context: Context,
    titulo: String,
    autor: String,
    paginas: String,
){
    val datos = "Titulo: $titulo Autor: $autor paginas: $paginas".trimIndent()

    context.openFileOutput(
        name = "datos.txt",
        mode = Context.MODE_PRIVATE
    ).use {
        archivo -> archivo.write( b = datos.toByteArray())
    }
}
```

Figura 10: Función Guardar Datos

Ejercicio 4: Al presionar “Ver registro”, leer el archivo guardado y mostrar su contenido por consola empleando la clase Log (Log.d).

Al presionar el botón “Ver registro” se ejecuta LeerDatos(context), que abre el archivo datos.txt con context.openFileInput("datos.txt"), lee su contenido mediante bufferedReader() y use { it.readText() }. El texto leído se muestra por consola empleando la clase Log con Log.d(tag = "REGISTRO-LIBRO", msg = datos).

```
fun LeerDatos(context: Context) {
    val datos = context.openFileInput("datos.txt")
        .bufferedReader()
        .use { it.readText() }

    Log.d(
        tag = "REGISTRO-LIBRO",
        msg = datos
    )
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```

    )
}

```

Al ejecutar el proyecto y presionar “Ver registro”, el contenido del archivo aparece en el Logcat con la etiqueta REGISTRO-LIBRO:

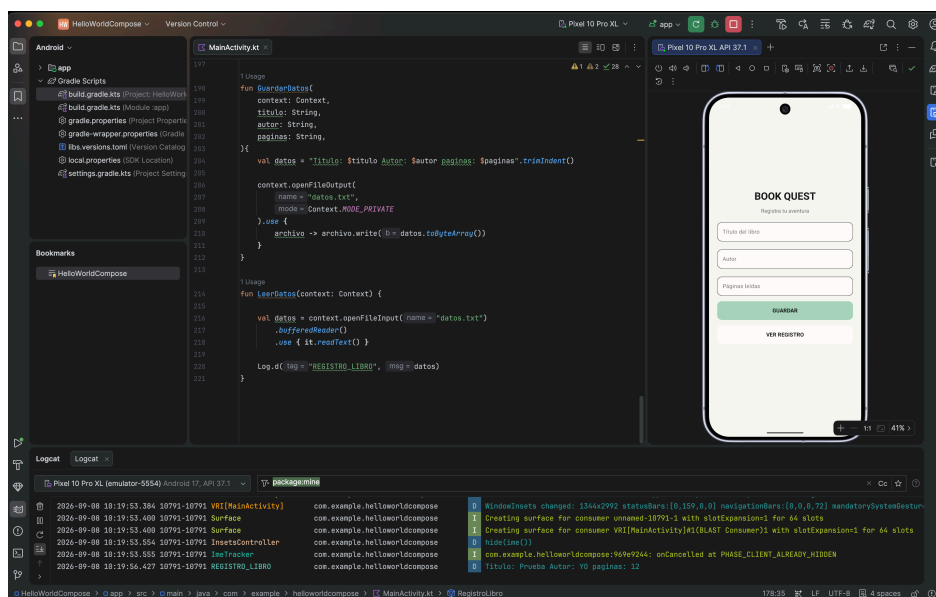


Figura 11: Ver Registro en consola

Ejercicio 5: (Reto opcional) Además de mostrarlo por consola, mostrar el registro leído en pantalla empleando un Text.

Al presionar el botón “Ver registro”, el resultado de LeerDatos(context) se guarda en la variable de estado registro. Con la condicional if (registro.isNotEmpty()) se muestra en pantalla el título “LIBRO REGISTRADO” y una tabla con los encabezados TÍTULO, AUTOR y PÁGINAS. Debajo, una segunda Row muestra los datos leídos (título, autor y paginas), dándoles el mismo peso con Modifier.weight(...).

```

if (registro.isNotEmpty()) {
    Text(
        text = "LIBRO REGISTRADO",
        fontSize = 18.sp,
        fontWeight = FontWeight.Bold,
        color = Black
    )

    Row(
        Modifier
            .fillMaxWidth()
            .background(Mint, RoundedCornerShape(12.dp))
            .padding(12.dp)
    ) {
        Text("TÍTULO", Modifier.weight(1.5f), fontWeight = FontWeight.Bold, color = Black)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```

Text("AUTOR", Modifier.weight(1.5f), fontWeight = FontWeight.Bold, color = Black)
Text("PÁGINAS", Modifier.weight(1f), fontWeight = FontWeight.Bold, color = Black)
}

Row(
    Modifier
        .fillMaxWidth()
        .background(Surface, RoundedCornerShape(12.dp))
        .padding(12.dp)
) {
    Text(titulo, Modifier.weight(1.5f), color = Black)
    Text(author, Modifier.weight(1.5f), color = Black)
    Text(paginas, Modifier.weight(1f), color = Black)
}
}

```

Al ejecutar el proyecto y presionar "Ver registro", el registro leído también se muestra en pantalla:

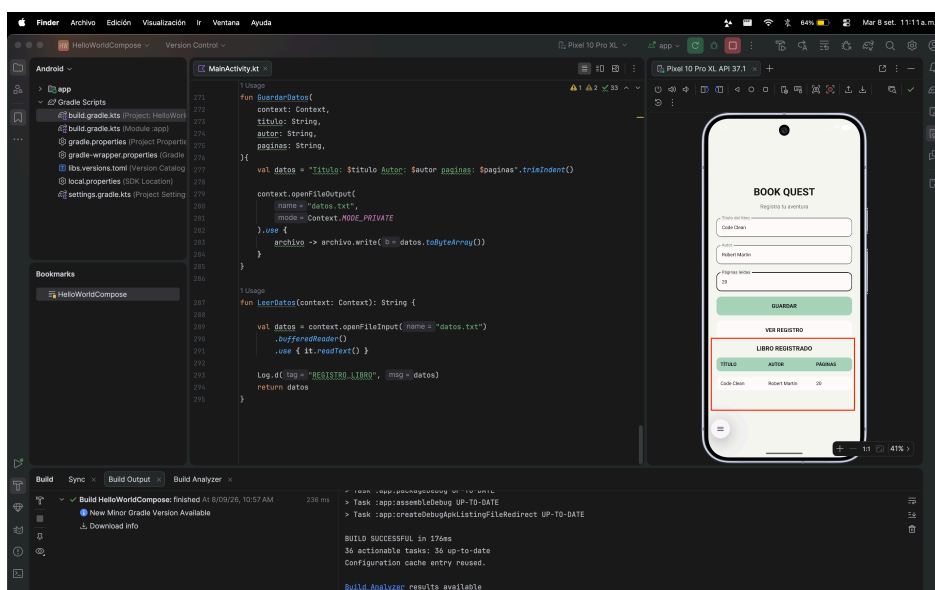


Figura 12: Reto Final: registro en pantalla

Propuesta 2

Ejercicio 1: Replicar el ejercicio resuelto por el docente (proyecto Compose + Hello World).

1. Se inició Android Studio y se creó un nuevo proyecto con la plantilla Empty Activity (Jetpack Compose) en lenguaje Kotlin, esperando a que Gradle finalice la sincronización

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

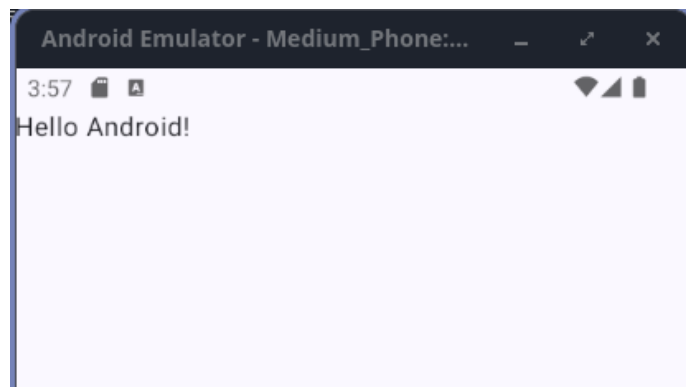


Figura 13: Hello Android

- En MainActivity.kt, se cambió el parámetro de la función Greeting dentro de setContent {} por mi nombre.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            HelloWorldComposeTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Denise",
                        modifier = Modifier.padding(paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}
```

Figura 14: Función Greeting

- Volver a ejecutar Run y verificar que el emulador ahora muestra el saludo personalizado.

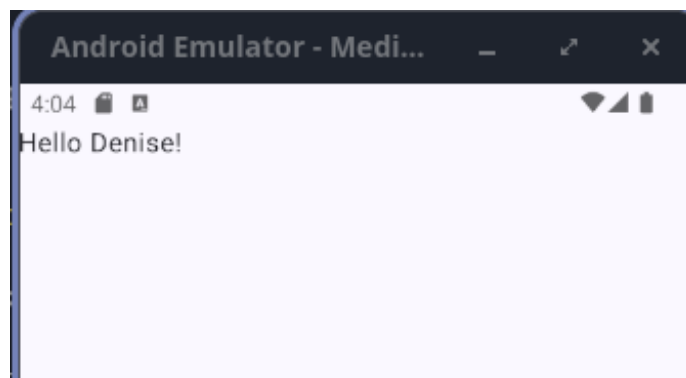


Figura 15: Hello Denise

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

Ejercicio 2: Una pantalla (Composable) con tres campos de texto (TextField): título del libro, autor y número de páginas leídas, y dos botones: "Guardar" y "Ver registro".

En este ejercicio, se implementó el composable RegistroLibroScreen en Jetpack Compose, donde se realizó una interfaz limpia y organizada para así capturar los datos requeridos (título, autor y páginas leídas) y los botones de acción ("Guardar" y "Ver registro").

1. Se declararon 3 variables de estado

Se empleó la función remember junto con mutableStateOf("") para definir tres variables de estado (título, autor y páginas).

```
// 1. Estado de los inputs (capturan lo que escribe el usuario)
var titulo by remember { mutableStateOf("") }
var autor by remember { mutableStateOf("") }
var paginas by remember { mutableStateOf("") }
```

2. Implementación de los campos de texto (OutlinedTextField)

Se utilizaron tres componentes OutlinedTextField organizados verticalmente dentro de un Column. Se agregó un ícono superior representativo (Icons.Default.Book).

```
Card(
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(12.dp),
    elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
) {
    Column(
        modifier = Modifier
            .padding(16.dp)
            .fillMaxWidth(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Icon(
            imageVector = Icons.Default.Book,
            contentDescription = "Libro",
            modifier = Modifier.size(48.dp)
        )
        OutlinedTextField(
            value = titulo,
            onValueChange = { titulo = it },
            label = { Text("Título del Libro") },
            modifier = Modifier.fillMaxWidth()
        )
        OutlinedTextField(
            value = autor,
            onValueChange = { autor = it },
            label = { Text("Autor") },
            modifier = Modifier.fillMaxWidth()
        )
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

}
OutlinedTextField(
  value = paginas,
  onValueChange = { paginas = it },
  label = { Text("Páginas leídas") },
  modifier = Modifier.fillMaxWidth()
)

```

3. Agregamos los botones de acción

Para las acciones requeridas se implementó una fila horizontal (Row) con `Arrangement.SpaceEvenly`, distribuyendo dos botones:

- **GUARDAR:** Implementado mediante un `Button` que se encargará de persistir la información ingresada.
- **VER REGISTRO:** Implementado con un `OutlinedButton` para la consulta y visualización de los datos guardados.

```

// Botones Guardar y Ver registro
Row(
  modifier = Modifier.fillMaxWidth(),
  horizontalArrangement = Arrangement.SpaceEvenly
) {
  Button(onClick = { /* Acción guardar */ }) {
    Text("GUARDAR")
  }
  OutlinedButton(onClick = { /* Acción ver registro */ }) {
    Text("VER REGISTRO")
  }
}

```

4. Resultado de la interfaz en el emulador

Al ejecutar la aplicación en el emulador de Android, se observa la interfaz completa con los tres campos de texto, el icono temático y los dos botones solicitados:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

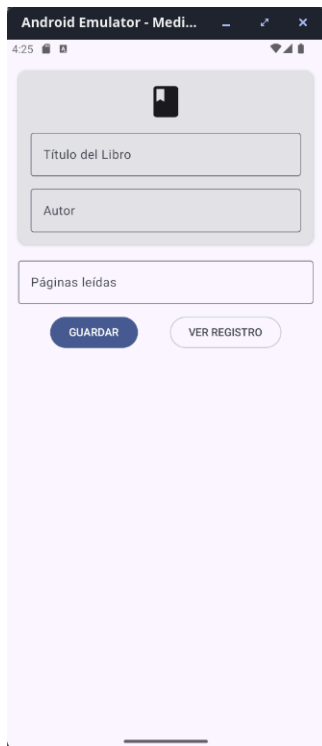


Figura 16: Pantalla con los tres campos de texto y botones de acción en Jetpack Compose

Ejercicio 3: Al presionar “Guardar”, escribir los tres datos ingresados en un archivo de texto plano guardado en el almacenamiento interno de la aplicación (openFileOutput, modo MODE_PRIVATE).

Para este ejercicio se implementó en la lógica asociada al botón “GUARDAR”. La solución se enfoca en validar los tres campos requeridos y persistirlos en un archivo de texto plano en el almacenamiento interno privado:

1. Escritura en almacenamiento interno (openFileOutput en modo MODE_PRIVATE)

El método abre o crea el archivo en el directorio interno de la aplicación. Mediante la función `.use`, se realiza la escritura de los bytes (`toByteArray()`) y se asegura el cierre del stream. La operación se protege dentro de un bloque try-catch, notificando el resultado al usuario mediante un Toast:

```
// Escritura en almacenamiento interno usando openFileOutput en modo MODE_PRIVATE
try {
    context.openFileOutput("registro_libro.txt", Context.MODE_PRIVATE).use { output ->
        output.write(datosAGuardar.toByteArray())
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

    }
    Toast.makeText(context, ";Datos guardados con éxito!", Toast.LENGTH_SHORT).show()
} catch (e: Exception) {
    e.printStackTrace()
    Toast.makeText(context, "Error al guardar el archivo", Toast.LENGTH_SHORT).show()
}

```

Ejercicio 4: Al presionar “Ver registro”, leer el archivo guardado y mostrar su contenido por consola empleando la clase Log (Log.d).

Para este ejercicio se implementó la funcionalidad asociada al botón “**VER REGISTRO**”. Donde se pueda abrir el archivo guardado en el almacenamiento interno privado, leerlo y enviarlo a la consola Logcat utilizando el método Log.d, además de gestionar posibles excepciones en caso de que el archivo aún no exista:

1. Lectura del archivo en almacenamiento interno (openFileInput) y salida por consola (Log.d)

Se utiliza el método openFileInput para abrir el archivo. Se realizó la lectura completa del contenido e imprimir el resultado en la consola mediante Log.d usando la etiqueta "RegistroLibro":

```

// Lectura del archivo desde el almacenamiento interno
val contenido = context.openFileInput("registro_libro.txt")
    .bufferedReader()
    .use { it.readText() }

// Imprimir en consola Logcat usando la clase Log (Log.d)
Log.d("RegistroLibro", "Contenido recuperado del archivo:\n$contenido")

```

2. Resultado de la consola

Se observa la consola después de presionar ver registro en nuestra interfaz:

```

com.example.helloworldcompose  D  Contenido recuperado del archivo:
                               Título: alicia en las maravillas
                               Autor: andrea h
                               Páginas: 23

```

Figura 17: Terminal con el registro del libro

Ejercicio 5: (Reto opcional) Además de mostrarlo por consola, mostrar el registro leído en pantalla empleando un Text.

Para el reto opcional, se implementó la visualización del archivo en pantalla utilizando componentes Text dentro de una tarjeta (Card):

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

1. Declaración de la variable de estado

Se definió la variable de estado `contenidoGuardado`, para mostrar u ocultar la tarjeta con el registro en tiempo real:

```
// Estado para almacenar y observar el contenido leído del archivo
var contenidoGuardado by remember { mutableStateOf("") }
```

2. Lectura del archivo y actualización del estado al presionar "VER REGISTRO"

En el botón "VER REGISTRO", se abre el archivo `registro_libro.txt` y se lee su contenido y se actualiza el estado `contenidoGuardado`, notificando a Compose que debe refrescar la vista con los nuevos datos:

```
// Lectura del archivo y actualización de la variable de estado
val contenido = context.openFileInput("registro_libro.txt")
    .bufferedReader()
    .use { it.readText() }
// Envío a la consola Logcat
Log.d("RegistroLibro", "Contenido recuperado del archivo:\n$contenido")
// Actualización del estado para reflejar los datos en pantalla
contenidoGuardado = contenido
```

3. Renderizado del registro en pantalla mediante componentes Text

Para presentar los datos se utiliza la condición `if (contenidoGuardado.isNotEmpty())`. Si existe contenido leído, se muestra una tarjeta Card. :

```
if (contenidoGuardado.isNotEmpty()) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant
        )
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(6.dp)
        ) {
            Text(
                text = "REGISTRO GUARDADO",
                fontWeight = FontWeight.Bold,
                fontSize = 14.sp,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
            HorizontalDivider(color = MaterialTheme.colorScheme.outlineVariant)
            Text(
                text = contenidoGuardado,
                fontSize = 14.sp,
            )
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

```

        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
}

```

4. Pruebas y visualización de resultados en el emulador

A continuación se muestra el flujo de funcionamiento de la aplicación:

1. **Guardado de los datos:** Se ingresan los datos del libro (título, autor y páginas leídas) y, al presionar **"GUARDAR"**, se guarda y se notifica al usuario mediante un mensaje Toast:



Figura 18: Persistencia de datos en almacenamiento interno y confirmación con Toast

2. **Visualización del registro en pantalla:** Al presionar **"VER REGISTRO"**, se realiza la lectura del archivo y el contenido se visualiza dinámicamente en pantalla mediante los componentes Text:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18



Figura 19: Visualización del registro leído en pantalla empleando componentes Text

CUESTIONARIO

II. CUESTIONARIO

Pregunta 1 : ¿Consideras complejo el entorno de Android Studio y el uso de Jetpack Compose para tu primera aplicación? Explica brevemente por qué.

No nos pareció complejo, el entorno de Android Studio facilitó bastante el flujo de trabajo tanto en la compilación como en la detección de errores. Aunque al principio me generó un poco de confusión por ser un lenguaje y un framework nuevos, la sintaxis de Kotlin resultó ser amigable y se logró comprender. Sobre el uso de Jetpack Compose, me pareció interesante y sencillo de usar, ya que permite crear interfaces de usuario de manera más fácil que el antiguo enfoque basado en XML.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

Pregunta 2 : ¿Cuál es el propósito de la función setContent {} dentro del onCreate() de una Activity que usa Jetpack Compose?

Primero una Activity es el componente que representa una pantalla individual con la que el usuario puede interactuar, y encargado de controlar su ciclo de vida. El método onCreate() es la primera función que se ejecuta cuando el sistema operativo crea la pantalla, es como un punto de entrada donde se realiza toda la configuración inicial.

Teniendo esto en cuenta, el propósito principal de setContent {} es ser el mecanismo que le dice a la Activity exactamente qué elementos gráficos debe dibujar en la pantalla. Actúa como el puente que conecta el ciclo de vida de la Activity con la interfaz gráfica. Antiguamente se utilizaba setContentView() que cargaba archivos XML.

Dentro del bloque de setContent {} se colocan las funciones que estructuran la pantalla, como textos, botones, listas o contenedores.

Ejemplo:

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            MiAplicacionTheme {
                GreetingScreen("Hola, Jetpack Compose") // Componente visual
            }
        }
    }
}
```

CONCLUSIONES

III. CONCLUSIONES Y RECOMENDACIONES

Conclusiones

Conclusión 1: Android Studio es una herramienta importante que simplifica el flujo de trabajo gracias a su integración con Gradle y herramientas de compilación en tiempo real.

Conclusión 2: Jetpack Compose sustituye de manera eficiente el diseño tradicional basado en XML, nos ayuda a construir interfaces de usuario mediante código Kotlin puro con una sintaxis más limpia y directa.

Conclusión 3: El uso de remember y mutableStateOf resulta muy útil en el desarrollo, ya que nos permite que la pantalla guarde lo que el usuario escribe y se actualice sola de forma automática, lo cual facilita mucho la creación de interfaces dinámicas. **Conclusión 4:** El uso de openFileOutput en modo Context.MODE_PRIVATE nos garantiza un mecanismo rápido y seguro para la persistencia de datos en el almacenamiento interno de la aplicación.

Conclusión 5: Las herramientas como Logcat y la renderización dinámica mediante componentes Text facilitan el diagnóstico y la verificación inmediata del flujo de datos en la pantalla.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 20

Recomendaciones

Recomendación 1: Implementar validaciones previas en los campos de texto antes de procesar el guardado, para asegurarnos que los datos no estén vacíos y tengan el formato adecuado.

Recomendación 2: Modularizar el proyecto separando los componentes `@Composable` en archivos Kotlin independientes nos ayuda a mantener un código estructurado y mantenible.

Recomendación 3: En lugar de guardar la información en un archivo de texto simple (.txt), se sugiere usar herramientas más avanzadas como Room o DataStore, ya que estos nos permiten organizar y proteger mucho mejor los datos de la aplicación.

Recomendación 4: Utilizar un dispositivo físico en caso de contar con recursos de hardware limitados para evitar la lentitud de los emuladores.

Recomendación 5: Incluir el control de excepciones (try-catch) al realizar la lectura de archivos para prevenir fallos inesperados si el usuario intenta consultar registros inexistentes.

REFERENCIAS

Bibliography

Android Developers. (2026). *Jetpack Compose basics* [Codelab]. <https://developer.android.com/codelabs/jetpack-compose-basics>

Android Developers. (2026). *Thinking in Compose*. <https://developer.android.com/develop/ui/compose/mental-model>

Android Developers. (2026). *Data and file storage overview*. <https://developer.android.com/training/data-storage>